

Les expressions régulières

M2 TNAH – 2026/2027

Selma Bensidhoum

selma.bensidhoum@mnhn.fr

Qu’est-ce qu’une expression régulière ?

Une expression régulière (*regex*) permet de décrire un ensemble de chaînes de caractères selon une syntaxe précise. Les usages sont multiples : rechercher, extraire, remplacer, valider…

Il ne s’agit pas d’un langage de programmation.

Les *regex* s’écrivent elles-mêmes sous la forme de chaînes de caractères.

La syntaxe peut varier légèrement selon les outils, nous allons nous concentrer sur le noyau commun.

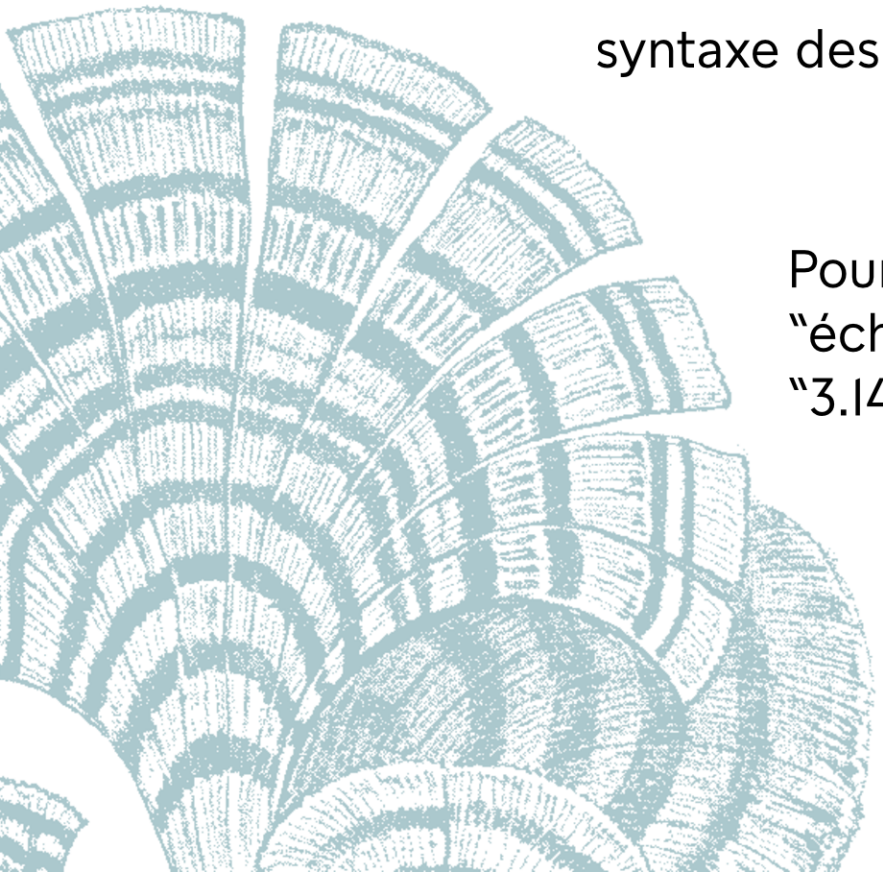
Ressources du cours

https://regex101.com/

https://github.com/SelmaKaina/Regex_TNAH/blob/main/Exercices.md

Caractères littéraux et métacaractères

Les caractères littéraux sont des caractères qui se représentent eux-mêmes. Une regex composée de caractères littéraux est une recherche “classique” : “chat” trouve “chat” mais pas “Chat”.



Les métacaractères sont des caractères avec une fonction particulière dans la syntaxe des regex : \ ^ $. | * ? + ( ) { } [ ]

Pour rechercher ces caractères comme des caractères littéraux, il faut les “échapper”, c’est-à-dire les précéder d’un backslash “\” : “3\.14” trouve “3.14”.

Le backslash est également utilisé pour désigner les caractères non-imprimables (ex. tabulation, retour à la ligne, espace…) et des raccourcis de classes de caractères.

Métacaractère	Rôle
.	Correspond à n'importe quel caractère (lettre, chiffre, symbole, espace...), sauf un retour à la ligne ou un saut de paragraphe.
*	L'élément qui précède peut être absent ou répété plusieurs fois (0 fois ou plus).
+	L'élément qui précède doit être présent I fois ou plus (l'élément est obligatoire, éventuellement répété).
?	L'élément qui précède peut être absent ou présent I fois (l'élément est facultatif).
|	Permet d'indiquer une alternative entre les éléments situés de part et d'autre de la barre verticale ("ou").

1. Cherchez “un”. Que constatez-vous ?

“un” ⇒ capture les articles mais également des éléments de “brun” et “jeune”

2. Trouver “lupa”, “lupi” et “lupo”.

“lup.” ⇒ donne 6 correspondances : lupa, lupi, lupo, ainsi que le début de lupos, de lupis et de lupus

3. Trouver tous les points et les virgules.

“\.|,” ⇒ le point doit être échappé, la barre verticale permet d’indiquer l’alternative

4. Remplacer “il” et “elle” par “iel”

Rechercher “il|elle”, remplacer par “iel”
⇒ le “il” de “Louis-Philippe” est aussi remplacé, alors que “Il” et “Elle” en début de phrase sont ignorés

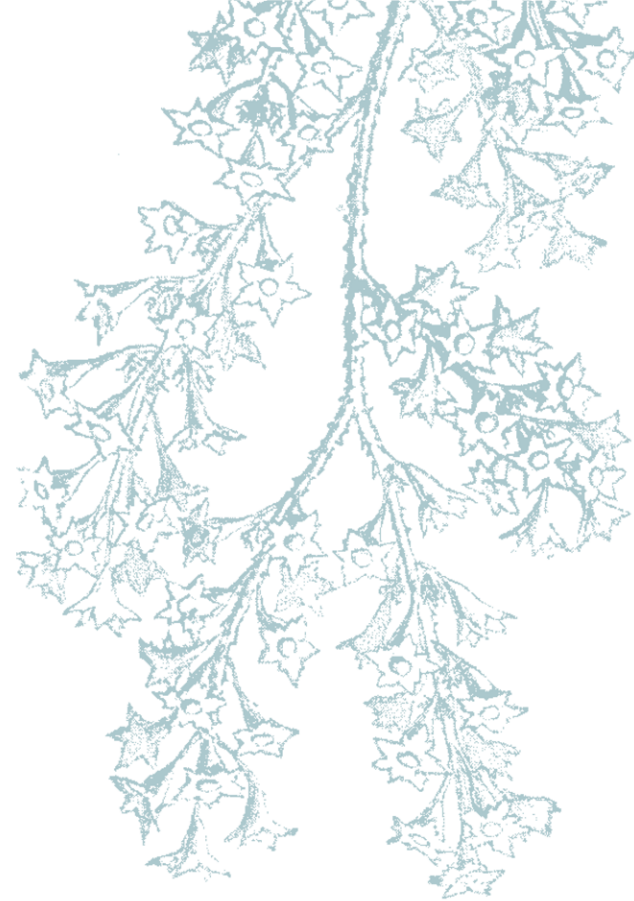
5. Corriger “2.5 kg” en “2,5 kg”

“2\.5” ⇒ “2.5” trouve deux résultats car le point renvoie à “tout caractère”

6. Trouver “(Pyrénées-Atlantiques)”

“\(Pyrénées-Atlantiques\)” ⇒ il faut échapper les parenthèses

Classes de caractères



Les listes

⇒ gr[ea]y trouve “grey” et “gray” mais pas “greay”
⇒ 1[34]92 trouve “1492” et “1392” mais pas “13493”

Les intervalles

⇒ [b-d]ar trouve “bar” et “dar” mais pas “par” ou “Car”
⇒ [2-5]0 trouve “20” et “30” et une partie de “150”

Les exclusions

Il est également possible de signaler les caractères qu’on ne veut pas trouver à cet endroit.
⇒ [^p]ain trouve “bain” et “sain” mais pas “pain”
⇒ 1[^0-7] trouve “18” et “19” mais pas les nombres entre 10 et 17

Les classes POSIX

Les classes POSIX sont des noms de classes de caractères définis par la norme POSIX, utilisables à la place des intervalles.

Classe	Utilisation	Raccourci	Classe POSIX
[A-Z]	Toute lettre majuscule		[[:upper:]]
[a-z]	Toute lettre minuscule		[[:lower:]]
[A-z]	Toute lettre minuscule ou majuscule		[[:alpha:]]
[a-zA-ZÀ-ÿ]	Toute lettre accentuée ou non, minuscule ou majuscule		
[0-9]	Tout caractère numérique	\d	[[:digit:]]
[^0-9]	Inverse de \d	\D	
[a-zA-Z0-9_]	Tout caractère alphanumérique et _	\w	[[:word:]]
[^a-zA-Z0-9_]	Inverse de \w	\W	
[\t\n\f\r]	Tout caractère d'espacement : espace, retour de chariot, tabulation..	\s	[[:space:]]
[!"#$%&'()*+,\-./:;⇔?@[\]^_`{|}~]	Tout caractère de ponctuation		[[:punct:]]

1. Trouver “marmotte” et “marmotto”

“marmott[eo]”

2. Trouver “lupos” et “lupis”, mais pas “lupus”.

“lup[oi]s” ou “lupos|lupis”

3. Trouver toutes les années.

“[0-9][0-9][0-9][0-9]” ou “\d\d\d\d”
⇒ “2 100” n’est pas trouvé à cause de l’espace

4. Trouver les années à partir de 2000.

“20[0-9][0-9]” ou “20\d\d”

5. Trouver les années antérieurs à 1800.

“1[0-7][0-9][0-9]” ou “1[0-7]\d\d”

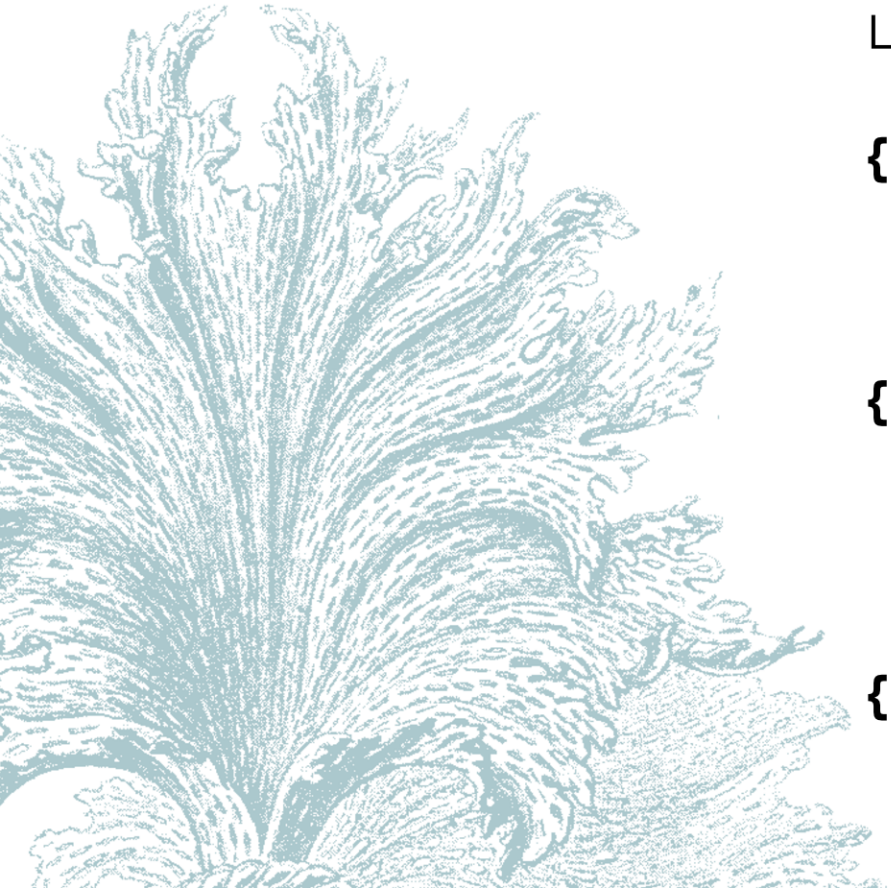
6. Trouver les années de 1700 à 1749.

“17[^5-9][0-9]” ou “17[^5-9]\d”

Quantificateurs

Par défaut, à part les groupes, un élément ne représente qu'un seul caractère. Les quantificateurs permettent d'indiquer combien de fois l'élément qui précède est répété.

Nous avons déjà vu plusieurs quantificateurs tels que : ? + et *



Les quantificateurs plus spécifiques se notent entre accolades :

{min,max}

co{2,3}l ⇒ "cool" ou "coool" mais pas "col" ou "cooool"
{0,1} est équivalent à ?

{min,}

[a-z]{2,} ⇒ "et" ou "anticonstitutionnel" mais pas "a"
{0,} est équivalent à *
{1,} est équivalent à +

{nombre}

\d{5} ⇒ "75004" ou "92000" mais pas "1789"

1. Trouver “grrr”, “grrrrr” et “grr”. Les remplacer par “ouille”.

“gr{2,}”

2. Trouver les années antérieures à 1900.

“1[0-8][0-9]{2}”

3. Trouver les prénoms composés.

“[A-Z][a-z]+-[A-Z][a-z]]+”
⇒ “[A-Z][a-zà-ÿ]+-[A-Z][a-zà-ÿ]]+” trouve aussi “Pyrénées-Atlantiques”

4. Trouver les mots de 15 lettres et plus.

“[a-zà-ÿ]{15,}”

5. Trouver les espaces doublées et les remplacer par une seule espace.

“ {2,}”

6. Trouver les mots qui commencent par deux majuscules.

“[A-Z]{2}[a-z]+”

Groupes

En plaçant une partie d'une regex entre parenthèses, on l'isole : on peut y appliquer des quantificateurs ou proposer une alternative. On parle alors de groupes. Par défaut, un groupe est capturant : il mémorise ce qu'il a trouvé. Il n'y a pas de limite au nombre de groupes, et on peut les imbriquer.

⇒ (James)?Bond trouve "Bond' et "James Bond" mais pas "James"

⇒ (Papaver (rhoeas|somniferum)) trouve "Papaver rhoeas" et "Papaver soniferum"

Chaque groupe reçoit automatiquement un numéro, selon la position de sa parenthèse ouvrante, de gauche à droite. Pour réutiliser son contenu, on fait référence à son numéro : $I dans LibreOffice, \I en Python.

Groupes non capturants
Pour inclure dans la recherche un élément qu'on ne souhaite pas le réutiliser, on peut utiliser un groupe non capturant (?:...) : il fait partie du motif, mais ce qu'il trouve n'est pas mémorisé. Par exemple, dans un texte contenant des civilités, la *regex* (?:M\.|Mme|Mlle) (\w+) repère uniquement les noms qui suivent une civilité.



1. Supprimer les civilités. Obtenir “Dubois, Martin et Bernard”.

“(\d{2})(\/|-)(\d{2})(\/|-)(\d{4})”
⇒ Remplacer par : “$5/$3/$l”

2. Transformer “Auteur : Camille Lenoir” en “Auteur : Lenoir, Camille”.

“Auteur : (\w+) (\w+)”
⇒ Remplacer par : “Auteur : $2, $1”

3. Trouver les mots répétés trois fois.

“\b([a-za-ÿ]+)(, |)\1(, |)\1\b”

4. Convertir l’URL en markdown.

“(https:\/\/[^]+)”
⇒ Remplacer par : “[lien]($1)”

5. Remplacer les guillemets droits en guillemets français.

“"([^"]+)"”
⇒ Remplacer par : “« $1 »”

6. Transformer la note [^ voir la fiche…] en <note>voir la fiche…</note>

“\[\^ ([^\]]+)\]”
⇒ Remplacer par : “<note>$1</note>”

7. Supprimer les civilités. Obtenir “Dubois, Martin et Bernard”.

“(?:M\.|Mme|Mlle) (\w+)”
⇒ Remplacer par : “$1”

Ancres

Ancres/ séquences	Rôle
^	L'élément qui suit se trouve en début de ligne/chaîne (ou de paragraphe dans LibreOffice).
$	L'élément qui précède se trouve en fin de ligne/chaîne (ou de paragraphe dans LibreOffice).
\n	Représente un retour à la ligne.
\R	Représente n'importe quel saut de ligne (absent dans le module re de Python).
\t	Représente une tabulation.
\b	Représente la limite d'un mot (début ou fin).
\B	L'élément se situe en milieu de mot.

1. Trouver tous les mots terminant par la lettre “c”.

2. Trouver tous les mots d’exactement 3 lettres.

3. Trouver tous les “c” en milieu de mot.

4. Trouver les minuscules en début de vers.

5. Trouver les vers qui se terminent par une virgule.

Recherche avant/arrière

Parfois, on cherche un élément qui en suit ou précède un autre. Pour cela, on utilise les fonctions de recherche avant et arrière (*lookahead* et *lookbehind*).

Par défaut, les *lookahead* et *lookbehind* ne sont pas capturés par la *regex*.

Lookahead

Le *lookahead* permet de capturer un élément en fonction de l'élément qui le suit.

- *Lookahead* positif : X(?=Y) ⇒ "rechercher X, mais seulement s'il est suivi de Y"
- *Lookahead* égatif : X(?!Y) ⇒ "rechercher X, mais seulement si il n'est pas suivi de Y"

Lookbehind

Le *lookbehind* permet de capturer un élément en fonction de l'élément qui le précède.

- *Lookbehind* positif: (?<=Y)X ⇒ "rechercher X, mais seulement s'il est précédé de Y"
- *Lookbehind* negatif: (?<!Y)X=> "rechercher X, mais seulement s'il n'est pas précédé de Y.

Exemples

\d+(?=€) ⇒ ne capture que les nombres entiers suivis du symbole €

(?<!Asplenium (trichomanes ⇒ ne captures que les occurrence de "trichomanes" précédées de "Asplenium"

1. Trouver les occurrences de "renard" suivi de "roux", sans capturer "roux".

2. Trouver les nombres qui ne sont pas suivis d'une unité, sans capturer l'unité.

3. Trouver les mots suivis d'une virgule, sans capturer la virgule.

4. Trouver "rusé" sauf dans l'expression "pas rusé".

5. Trouver le mois de la date "12-05-2021".

6. Extraire le texte de la note de bas de page sans les délimiteurs, puis le texte entre guillemets sans capturer les guillemets.